# Wiki: Automated Testing Framework Documentation

# Overview

This automated testing framework is designed for testing the functionality of a web application. It utilizes Selenium WebDriver for browser automation and Cucumber for behavior-driven development (BDD) testing. The framework supports multiple browser options, handles cookies, and provides utility functions for interacting with web elements. We'll cover the project structure, utility methods, best practices, and provide an example of automating interactions with the Learnmore page.

# Project Structure

The project is organized into different packages and files:

- `pom.xml`: The Maven POM file contains project metadata, dependencies, and configuration settings.
- `utils` Package: Contains utility classes for WebDriver management and browser configuration.
- `DriverFactory` Class: Provides methods to create and manage WebDriver instances.
- `testRunner` Package: Includes the Cucumber test runner class and test setup.
- `SetupTest` Class: Manages test setup, browser configuration, and cookie handling.
- `TestRunner` Class: Configures and executes Cucumber tests.
- `stepDefinitions` Package: Contains Cucumber step definitions for scenarios.
- `LearnmoreSteps` Class: Implements step definitions for automating Learnmore page interactions.
- `pages` Package: Houses page object classes representing different pages of the website.
- `Learnmore` Class: Provides methods to interact with web elements on the Learnmore page.

# Utility Methods

### `DriverFactory` Class (utils.DriverFactory)

- `getDriver(String browserName)`: Returns a WebDriver instance based on the provided browser name.
- `quitDriver()`: Closes the WebDriver instance and releases resources.

### `SetupTest` Class (testRunner.SetupTest)

- `configureBrowser()`: Retrieves the browser name from config.properties.
- `setURL()`: Retrieves the URL from config.properties.
- `deleteAllCookiesWithSafariWorkaround(WebDriver driver)`: Deletes cookies, with a workaround for SafariDriver.
- `configureTestRunner()`: Sets up the test runner with browser configuration and cookie management.
- `clickAcceptAllCookiesButton(WebDriver driver)`: Clicks the "Accept All Cookies" button.
- `clickCloseCookiesButton(WebDriver driver)`: Clicks the "Close Cookies" button.
- `getProperty(String propertyName)`: Reads data from config.properties.

# Configuration File

The framework uses a configuration file named `config.properties` to store important settings.

Example `config.properties`:

```
browserName=chrome
url=http://localhost:3000/learnmore
tags=@test1
```

- `browserName`: Specifies the browser to be used for testing (e.g., `chrome`, `firefox`, `edge`, `safari`).
- `url`: Defines the base URL of the web application to be tested.
- `tags`: Sets the tags for scenarios to be executed during test runs.

# Page Objects

### `Learnmore` Class (pages.Learnmore)

This page object class represents the Learnmore page of the website and provides methods to interact with its web elements.

- `isVideoThumbnailDisplayed()`: Checks if the video thumbnail is displayed.
- `clickPlayVideoButton()`: Clicks the "Play Video" button.
- `isVideoPlaying()`: Checks if the video is currently playing.

# TestRunner Class

The `TestRunner` class (`testRunner.TestRunner`) is the entry point for executing Cucumber tests. It is responsible for setting up the test environment, configuring the test runner, and executing the specified scenarios. Let's take a closer look at its key components:

- `@RunWith(Cucumber.class)` Annotation: This annotation is used to specify that Cucumber should run the tests defined in this class. It indicates that the class serves as a test runner.
- `@CucumberOptions` Annotation: This annotation is used to configure various options for Cucumber test execution. It specifies features, glue code location, plugin options, and more.
- `@BeforeClass` Method: The `setup()` method is annotated with `@BeforeClass`, which means it will be executed before any test scenarios. It sets up the test environment, configures the browser, and initializes the WebDriver instance.
- `@AfterClass` Method: The `teardown()` method is annotated with `@AfterClass`, which means it will be executed after all test scenarios are completed. It closes the WebDriver instance and releases resources.
- `main(String[] args)` Method: This method reads tags from the `config.properties` file, sets the tags in Cucumber options, and then runs Cucumber tests using the `io.cucumber.core.cli.Main` class.

# `pom.xml` File

The `pom.xml` file is a critical component of the project, as it manages project dependencies, configurations, and builds. It uses Maven, a popular build automation tool, to ensure a consistent environment for building and running tests. Key sections in the `pom.xml` file include:

- `<dependencies>`: This section lists the project's dependencies, including Selenium WebDriver, Cucumber, JUnit, and any other required libraries. Dependencies ensure that the required components are available during the test execution.
- `<plugins>`: Maven plugins define tasks to be executed during the build lifecycle. Plugins like `maven-compiler-plugin` and `maven-surefire-plugin` are used to compile Java code and execute tests, respectively.
- `<properties>`: Properties allow you to define and reuse configuration values throughout the `pom.xml` file. In your project, it specifies the Java version and other settings.

# Best Practices

- **Modular Page Objects**: Organize page object classes by functionality or page to encapsulate interactions.
- **Reusable Steps**: Create reusable step definitions shared across scenarios for improved maintenance.
- **Parameterization**: Utilize data tables and scenario outlines to parameterize scenarios.
- **Assertions and Verification**: Use assertions to verify expected outcomes using tools like JUnit's `Assert`.
- **Naming Conventions**: Follow consistent and descriptive naming conventions for better code readability.
- **Logging and Reporting**: Integrate logging and generate comprehensive test reports.

# Example: Automating Learnmore Page

## Feature File (learnmore.feature)

```
Feature: Play Video on Learnmore Page

@test1

Scenario: Play video

Given I am on the learnmore page
When I locate the video element
And I click on the play button
Then the video should start playing
```

| Step Definitions (LearnmoreSteps.java) | Page Object (Learnmore.java) |
|---|---|
| Given("I am on the learnmore page")<br>public void i_am_on_the_learnmore_page() {<br>  // Implementation<br>}<br><br>When("I locate the video element")<br>public void i_locate_the_video_element() {<br>  // Implementation<br>}<br><br>When("I click on the play button")<br>public void i_click_on_the_play_button() {<br>  // Implementation<br>}<br><br>Then("the video should start playing")<br>public void the_video_should_start_playing() {<br>  // Implementation<br>} | public class Learnmore {<br><br>  public boolean isVideoThumbnailDisplayed() {<br>  // Implementation<br>  }<br><br>  public void clickPlayVideoButton() {<br>  // Implementation<br>  }<br><br>  public boolean isVideoPlaying() {<br>  // Implementation<br>  }<br>} |

# Conclusion

The automated testing framework, built using Selenium WebDriver and Cucumber, provides a structured approach to web application testing. By following the project structure, utility methods, best practices, and example provided in this guide, you can create efficient and maintainable automated tests. The use of the page object pattern enhances test readability, reusability, and organization, making it easier to manage complex test scenarios across different pages of your web application.

Customize the framework and example to match your application's specific requirements and elements. Use this guide as a foundation to create robust and reliable automated tests for your web application's functionality.